

ZAMAN UYUMSUZ PROGRAMLAMA

Zaman uyumsuz (**asynchron**) programlama, bir programın birden fazla işlemi aynı anda (eşzamanlı olarak) yapabilmesini sağlar. Python'da asenkron programlama için **asyncio** kütüphanesi kullanılır. Bu özellik Python 3.4'te tanıtılmış, Python 3.5'te **async/await** talimatları da eklenmiştir.

Zaman Uyumlu ve Uyumsuz Programlama

Zaman Uyumlu (**sincron**) programlamada, her işlem bir önceki işlemin bitmesini bekler;

```
import time

def gorev1():
    print("Görev 1 başladı")
    time.sleep(2) # 2 saniye bekle
    print("Görev 1 bitti")

def gorev2():
    print("Görev 2 başladı")
    time.sleep(1) # 1 saniye bekle
    print("Görev 2 bitti")

baslangic = time.time()
gorev1()
# Çıktı: Görev 1 başladı
# Çıktı: Görev 1 bitti
gorev2()
# Çıktı: Görev 2 başladı
# Çıktı: Görev 2 bitti
print(f"Toplam süre: {time.time() - baslangic:.2f} saniye")
# Çıktı: Toplam süre: 3.00 saniye
```

Zaman Uyumsuz (**asynchron**) programlamada ise görevler beklerken diğer görevler çalışabilir;

```
import asyncio

async def gorev1():
    print("Görev 1 başladı")
    await asyncio.sleep(2) # 2 saniye bekle (asenkron)
    print("Görev 1 bitti")

async def gorev2():
    print("Görev 2 başladı")
    await asyncio.sleep(1) # 1 saniye bekle (asenkron)
    print("Görev 2 bitti")

async def main():
    baslangic = time.time()
    await asyncio.gather(gorev1(), gorev2())
    print(f"Toplam süre: {time.time() - baslangic:.2f} saniye")

asyncio.run(main())
# Çıktı: Görev 1 başladı
# Çıktı: Görev 2 başladı
# Çıktı: Görev 2 bitti
# Çıktı: Görev 1 bitti
# Çıktı: Toplam süre: 2.00 saniye
```

Görüldüğü gibi asenkron programlamada toplam süre 2 saniye oldu çünkü görevler paralel çalıştı.

async ve await Anahtar Kelimeleri

Bu kelimeler zaman uyumsuz programlama için olmazsa olmazlardır;

async: Bir fonksiyonu zaman uyumsuz (**coroutine**) yapar.

await: Asenkron bir işlemin tamamlanmasını bekler, beklerken diğer görevlerin çalışmasına izin verir.

```
async def selam_ver(isim: str, bekleme: int):
    print(f"{isim} için bekleniyor...")
    await asyncio.sleep(bekleme)
    print(f"Merhaba {isim}!")
    return f"{isim} selamlandı"

async def main():
    sonuc = await selam_ver("Ali", 2)
    print(sonuc)

asyncio.run(main())
# Çıktı: Ali için bekleniyor...
# Çıktı: Merhaba Ali!
# Çıktı: Ali selamlandı
```

Birden Fazla Görevi Birlikte Çalıştırma

Bu kütüphanedeki **gather()** fonksiyonu ile birden fazla asenkron görevi aynı anda başlatabilir ve hepsinin bitmesini bekleyebiliriz:

```
async def indir(dosya: str, sure: int):
    print(f"{dosya} indiriliyor...")
    await asyncio.sleep(sure)
    print(f"{dosya} indirildi!")
    return f"{dosya} hazır"

async def main():
    sonuclar = await asyncio.gather(
        indir("resim1.jpg", 2),
        indir("resim2.jpg", 3),
        indir("resim3.jpg", 1)
    )
    print("Tüm indirmeler tamamlandı:")
    for sonuc in sonuclar:
        print(f" - {sonuc}")

asyncio.run(main())
# Çıktı: resim1.jpg indiriliyor...
# Çıktı: resim2.jpg indiriliyor...
# Çıktı: resim3.jpg indiriliyor...
# Çıktı: resim3.jpg indirildi!
# Çıktı: resim1.jpg indirildi!
# Çıktı: resim2.jpg indirildi!
# Çıktı: Tüm indirmeler tamamlandı:
# Çıktı: - resim1.jpg hazır
# Çıktı: - resim2.jpg hazır
# Çıktı: - resim3.jpg hazır
```

Görev Oluşturma

Zaman uyumsuz bir görev, **create_task()** ile bir coroutine'i **Task** nesnesine dönüştürüp arka planda çalıştırabiliriz:

```
async def sayac(isim: str, sayi: int):
```

```
for i in range(sayi):
    print(f"{isim}: {i+1}")
    await asyncio.sleep(1)
return f"{isim} tamamlandı"

async def main():
    # Görevleri oluştur (hemen başlatılır)
    gorev1 = asyncio.create_task(sayac("A", 3))
    gorev2 = asyncio.create_task(sayac("B", 4))

    # Her iki görevin tamamlanmasını bekle
    sonuc1 = await gorev1
    sonuc2 = await gorev2

    print(sonuc1)
    print(sonuc2)

asyncio.run(main())
# Çıktı: A: 1
# Çıktı: B: 1
# Çıktı: A: 2
# Çıktı: B: 2
# Çıktı: A: 3
# Çıktı: B: 3
# Çıktı: B: 4
# Çıktı: A tamamlandı
# Çıktı: B tamamlandı
```

Asenkron Dosya İşlemleri

Normal dosya işlemleri **zaman uyumludur** (**sincron**) ve programı bloke eder. **aiofiles** kütüphanesi ile **zaman uyumsuz** (**asynchron**) dosya işlemleri yapabiliriz:

```
import aiofiles

async def dosya_yaz(dosya_adi: str, icerik: str):
    async with aiofiles.open(dosya_adi, 'w', encoding='utf-8') as f:
        await f.write(icerik)
    print(f"{dosya_adi} yazıldı")

async def dosya_oku(dosya_adi: str) -> str:
    async with aiofiles.open(dosya_adi, 'r', encoding='utf-8') as f:
        icerik = await f.read()
    print(f"{dosya_adi} okundu")
    return icerik

async def main():
    # Paralel yazma
    await asyncio.gather(
        dosya_yaz("dosya1.txt", "İçerik 1"),
        dosya_yaz("dosya2.txt", "İçerik 2"),
        dosya_yaz("dosya3.txt", "İçerik 3")
    )

    # Paralel okuma
    sonuclar = await asyncio.gather(
        dosya_oku("dosya1.txt"),
        dosya_oku("dosya2.txt"),
        dosya_oku("dosya3.txt")
    )

    for i, icerik in enumerate(sonuclar, 1):
```

```
print(f"Dosya {i}: {icerik}")
```

```
# asyncio.run(main())
```

Asenkron HTTP İstekleri

Web istekleri genellikle yavaştır. `aiohttp` ile birden fazla isteği aynı anda yapabiliriz:

```
import aiohttp

async def sayfa_getir(url: str):
    async with aiohttp.ClientSession() as oturum:
        async with oturum.get(url) as yanit:
            icerik = await yanit.text()
            print(f"{url}: {len(icerik)} karakter")
            return len(icerik)

async def main():
    urllar = [
        "https://python.org",
        "https://github.com",
        "https://stackoverflow.com"
    ]

    sonuclar = await asyncio.gather(*[sayfa_getir(url) for url in urllar])

    print(f"Toplam: {sum(sonuclar)} karakter")

# asyncio.run(main())
```

Asenkron Bağlam Yöneticileri

Asenkron **bağlam yöneticileri** (**context manager**) `async with` kullanılarak tanımlanır:

```
class VeriTabani:
    async def __aenter__(self):
        print("Veritabanına bağlanılıyor...")
        await asyncio.sleep(1)
        print("Bağlantı kuruldu")
        return self

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        print("Bağlantı kapatılıyor...")
        await asyncio.sleep(0.5)
        print("Bağlantı kapatıldı")

    async def sorgu_calistir(self, sorgu: str):
        print(f"Sorgu çalıştırılıyor: {sorgu}")
        await asyncio.sleep(0.5)
        return f"Sonuç: {sorgu}"

async def main():
    async with VeriTabani() as vt:
        sonuc = await vt.sorgu_calistir("SELECT * FROM users")
        print(sonuc)

asyncio.run(main())
# Çıktı: Veritabanına bağlanılıyor...
# Çıktı: Bağlantı kuruldu
# Çıktı: Sorgu çalıştırılıyor: SELECT * FROM users
# Çıktı: Sonuç: SELECT * FROM users
# Çıktı: Bağlantı kapatılıyor...
```

```
# Çıktı: Bağlantı kapatıldı
```

Asenkron Yineleyiciler

Asenkron **yineleyiciler** (**iterator**) için `async for` ile kullanılır:

```
class AsenkronSayac:
    def __init__(self, baslangic: int, bitis: int):
        self.mevcut = baslangic
        self.bitis = bitis

    def __aiter__(self):
        return self

    async def __anext__(self):
        if self.mevcut >= self.bitis:
            raise StopAsyncIteration

        await asyncio.sleep(0.5) # Her sayıda 0.5 saniye bekle
        self.mevcut += 1
        return self.mevcut - 1

async def main():
    async for sayi in AsenkronSayac(1, 6):
        print(f"Sayı: {sayi}")

asyncio.run(main())
# Çıktı: Sayı: 1
# Çıktı: Sayı: 2
# Çıktı: Sayı: 3
# Çıktı: Sayı: 4
# Çıktı: Sayı: 5
```

Zaman Aşımı

Bir görevin belirli bir sürede tamamlanmasını zorlamak için `wait_for()` kullanılır:

```
async def uzun_islem():
    print("Uzun işlem başladı")
    await asyncio.sleep(5)
    print("Uzun işlem bitti")
    return "Tamamlandı"

async def main():
    try:
        sonuc = await asyncio.wait_for(uzun_islem(), timeout=2.0)
        print(sonuc)
    except asyncio.TimeoutError:
        print("İşlem zaman aşımına uğradı!")

asyncio.run(main())
# Çıktı: Uzun işlem başladı
# Çıktı: İşlem zaman aşımına uğradı!
```

Asenkron Kuyruk

Üretici-tüketici desen (**producer-consumer pattern**) için asenkron kuyruk kullanılır:

```
async def uretici(kuyruk: asyncio.Queue, urun_sayisi: int):
    for i in range(urun_sayisi):
        urun = f"Ürün-{i+1}"
        await kuyruk.put(urun)
```

```
    print(f"Üretildi: {urun}")
    await asyncio.sleep(0.5)

# Tüketicilere bittiği sinyali
await kuyruk.put(None)

async def tuketici(kuyruk: asyncio.Queue, isim: str):
    while True:
        urun = await kuyruk.get()
        if urun is None:
            # Bittiği sinyali alındı, diğer tüketiciler için geri koy
            await kuyruk.put(None)
            break

        print(f"{isim} işledi: {urun}")
        await asyncio.sleep(1)
        kuyruk.task_done()

async def main():
    kuyruk = asyncio.Queue()

    uretici_gorev = asyncio.create_task(uretici(kuyruk, 5))
    tuketici_gorevleri = [
        asyncio.create_task(tuketici(kuyruk, "Tüketici-1")),
        asyncio.create_task(tuketici(kuyruk, "Tüketici-2"))
    ]

    await uretici_gorev
    await asyncio.gather(*tuketici_gorevleri)

# asyncio.run(main())
```

Hata Yönetimi

Asenkron fonksiyonlarda da normal `try-except` kullanılır:

```
async def hata_olustur():
    await asyncio.sleep(1)
    raise ValueError("Bir hata oluştu!")

async def guvenli_calistir():
    try:
        await hata_olustur()
    except ValueError as e:
        print(f"Hata yakalandı: {e}")
    finally:
        print("Temizlik işlemleri yapıldı")

asyncio.run(guvenli_calistir())
# Çıktı: Hata yakalandı: Bir hata oluştu!
# Çıktı: Temizlik işlemleri yapıldı
```

İlk Tamamlanan

Birden fazla görevden ilk tamamlananı işlemek için `as_completed()` fonksiyonu kullanılır:

```
async def gorev(isim: str, sure: int):
    await asyncio.sleep(sure)
    return f"{isim} tamamlandı ({sure}s)"

async def main():
    gorevler = [
```

```
        gorev("Hızlı", 1),
        gorev("Orta", 2),
        gorev("Yavaş", 3)
    ]

    for tamamlanan in asyncio.as_completed(gorevler):
        sonuc = await tamamlanan
        print(sonuc)

asyncio.run(main())
# Çıktı: Hızlı tamamlandı (1s)
# Çıktı: Orta tamamlandı (2s)
# Çıktı: Yavaş tamamlandı (3s)
```

Özet ve En İyi Uygulamalar

- Girdi/çıkıtlı yoğun işlemler için `asyncio` idealdir
- İşlemci yoğun kullanılacağı işlemler (**CPU-bound**) için `multiprocessing` kullanılır
- `await` kullanmadan `async` fonksiyon çağrılmaz
- Bloke eden fonksiyonlar asenkron blokta kullanılmaz
- `time.sleep()` yerine `await asyncio.sleep()` kullanılır
- Uzun süren işlemler için `asyncio.wait_for()` ile zaman aşımı (**timeout**) eklenir
- Birden fazla görev için `gather()` veya `create_task()` kullanılır
- Zaman uyumsuz işlemler için asenkron kütüphaneler kullanılır (`aiohttp`, `aiofiles` vb.)
- Olay döngüsü (**event loop**), `asyncio.run()` ile başlatılır
- Hata yönetimi için `try-except` kullanılır
- Bağlam Yönetici (**context manager**) için `async with` kullanılır
- Yineleyiciler (**iterator**) için `async for` kullanın